Database Dictionary - Sirius Skool

Quick Reference

Table	Why It Exists	Business Problem It Solves	What Happens If You Don't Use It?	Importance
platform_admins	Stores Platform (Super) Admin accounts. Completely separate from school users.	Separates platform management from school management and provides an independent authentication system.	Platform Admin logic gets mixed with school users, making authentication, authorization, and future billing/features harder to maintain.	⭐⭐⭐⭐⭐
tenants	Represents each school (tenant) using the SaaS.	Enables multi-tenancy and isolates each school's data.	Impossible to build a true multi-tenant SaaS. All schools would share the same data.	⭐⭐⭐⭐⭐
tenant_settings	Stores school-specific settings (logo, contact info, SMS, email, branding, etc.).	Keeps configurable settings separate from business data.	Settings become scattered across multiple tables or hardcoded in the application.	⭐⭐⭐⭐☆
tenant_modules	Stores which modules are allocated by the Platform Admin and enabled by the School Admin.	Enables module licensing, feature toggles, and future subscription plans.	Module management becomes difficult and requires redesign later.	⭐⭐⭐⭐☆
users	Stores School Admin and Manager accounts.	Handles authentication and user management for each school.	No school login system or user management.	⭐⭐⭐⭐⭐
manager_permissions	Stores module-level CRUD permissions for each Manager.	Allows School Admins to assign different permissions to different Managers.	Permission logic becomes hardcoded or stored in JSON, making it harder to maintain and query.	⭐⭐⭐⭐☆
academic_years	Represents academic sessions (e.g., 2026–2027).	Preserves yearly academic history, promotions, attendance, and results.	Promotions overwrite previous academic information, making historical records unreliable.	⭐⭐⭐⭐⭐
classes	Stores class definitions (Nursery, KG, Class 1–10, etc.).	Allows each school to define its own class structure.	Classes become hardcoded and inflexible.	⭐⭐⭐⭐⭐
sections	Stores sections within classes (A, B, C, etc.).	Supports multiple sections and roll number uniqueness within a class.	Difficult to organize students and manage section-based operations.	⭐⭐⭐⭐⭐

Table	Why It Exists	Business Problem It Solves	What Happens If You Don't Use It?	Importance
subjects	Stores the master list of available subjects.	Centralizes subject management.	Subject information becomes duplicated or hardcoded.	⭐⭐⭐⭐⭐
class_subjects	Assigns subjects to specific classes.	*(Merged into `subjects` table via `class_id` column)*	*(No separate table needed)*	⬇️ Merged
applications	Stores admission applications before approval.	Separates applicants from enrolled students and supports the admission workflow.	Applicants and enrolled students become mixed together, making admissions difficult to manage.	⭐⭐⭐⭐☆
students	Stores each student's permanent identity and registration number.	Keeps permanent student information separate from yearly academic information.	Student history is lost when promoted because yearly data overwrites permanent records.	⭐⭐⭐⭐⭐
student_enrollments	Stores a student's class, section, roll number, academic year, and enrollment status.	Maintains complete academic history and simplifies student promotion.	Promotion overwrites class, section, and roll information, breaking historical data.	⭐⭐⭐⭐⭐
attendance_sessions	Represents one attendance-taking event (e.g., Class 6A on 10 July).	Stores session-level metadata (taken by, submitted at, locked status) and avoids data duplication.	Attendance metadata must be duplicated for every student record, making updates and auditing difficult.	⭐⭐⭐⭐☆
attendance_records	Stores each student's attendance status for an attendance session.	Tracks daily attendance and supports attendance reports.	Attendance feature cannot exist.	⭐⭐⭐⭐⭐
exams	Stores exam definitions (Midterm, Final, Monthly Test, etc.).	Organizes marks and results by examination.	Impossible to separate marks for different exams.	⭐⭐⭐⭐⭐
exam_subjects	Assigns subjects to an exam for a specific class and stores Full Marks, Pass Marks, etc.	Allows different classes to have different exam structures and grading rules.	Full marks, pass marks, and subject configurations become duplicated or hardcoded.	⭐⭐⭐⭐⭐
marks	Stores marks obtained by students for each exam subject.	Forms the foundation of the Result Management system.	Result generation becomes impossible.	⭐⭐⭐⭐⭐
grade_scales	Stores configurable grading rules (A+, A, GPA ranges, etc.).	Allows schools to customize grading without changing application code.	Grading logic becomes hardcoded and difficult to maintain.	⭐⭐⭐⭐☆
notices	Stores school notices.	Powers the Notice Board module.	Notice Board feature cannot exist.	⭐⭐⭐⭐☆
notifications	Stores SMS/Email notification requests, delivery status, retry	Supports asynchronous notifications, retries, and delivery tracking.	Notification processing becomes tightly coupled to attendance/result publishing,	⭐⭐⭐⭐☆

Table	Why It Exists	Business Problem It Solves	What Happens If You Don't Use It?	Importance
	count, and provider responses.		making failures difficult to recover from.	
audit_logs	Stores critical system activities (Create, Update, Delete, Publish, Permission Changes, etc.).	Provides accountability, security, and debugging.	Impossible to determine who changed or deleted important data.	⭐⭐⭐⭐⭐

Architecture Summary

Category	Tables
Platform	platform_admins
Core & Multi-Tenant	tenants, tenant_settings, tenant_modules
Authentication & Authorization	users, manager_permissions
Academic Structure	academic_years, classes, sections, subjects, ~~class_subjects~~
Student Management	applications, students, student_enrollments
Attendance	attendance_sessions, attendance_records
Examination & Results	exams, exam_subjects, marks, grade_scales
Communication	notices, notifications
System	audit_logs

Final Statistics

Item	Count
Platform Tables	1
Core & Multi-Tenant Tables	3
Authentication & Authorization Tables	2
Academic Structure Tables	4
Student Management Tables	3
Attendance Tables	2
Examination & Results Tables	4
Communication Tables	2
System Tables	1
Total Tables	**22**

Architecture Principles

- ✅ Multi-Tenant SaaS Ready
- ✅ Platform Admin & School Users Fully Separated
- ✅ Normalized Database Design (3NF)
- ✅ Supports Complete Student Academic History
- ✅ Future-Proof Result Management
- ✅ Flexible Module Licensing
- ✅ Fine-Grained Manager Permissions
- ✅ Scalable Notification System
- ✅ Audit & Accountability Support
- ✅ Designed for Future Expansion (Fees, Library, Hostel, Transport, Timetable, etc.)

Database Design Standards (Global Rules)

> **Project:** Sirius Skool - School Management SaaS
>
> These standards apply to the entire database unless explicitly stated otherwise. Every table should follow these conventions to ensure consistency, scalability, and maintainability.

---

1. Database Engine

- **Database:** PostgreSQL
- **Primary Key Type:** UUID
- **UUID Generation:** `gen_random_uuid()` (pgcrypto)

---

2. Naming Convention

Tables

- Use **plural** names.
- Use **snake_case**.

✅ Good

```
students
users
academic_years
student_enrollments
```

❌ Bad

```
student
Student
StudentTable
```

Columns

- Use **snake_case**
- Foreign keys should always end with `_id`

✅ Good

```
student_id
tenant_id
exam_id
created_at
```

❌ Bad

```
StudentID
sid
StudentId
```

---

3. Primary Keys

Every table must contain:

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
```

Reasons:

- Globally unique
- Safe for APIs
- Better for multi-tenant architecture
- Easier future migrations

- Prevents predictable IDs

---

4. Foreign Keys

- Always reference UUID primary keys.
- Always use descriptive names.

Example:

```
 tenant_id
student_id
class_id
user_id
exam_id
```

Never:

```
 tid
sid
cid
```

---

5. Timestamp Columns

Unless there is a strong reason not to, every table should contain:

```
 created_at TIMESTAMPTZ DEFAULT NOW()

updated_at TIMESTAMPTZ DEFAULT NOW()
```

Reasons:

- Audit trail
- Debugging
- Sorting
- Reporting

---

6. Time Zone

Always use

```
TIMESTAMPTZ
```

Never use plain

```
TIMESTAMP
```

This ensures correct handling across different time zones.

---

7. Multi-Tenant Rule

Every table that belongs to a school (tenant) must contain:

```
tenant_id
```

Exceptions:

- platform_admins
- tenants

This provides:

- Strong tenant isolation
- Simpler queries
- Better indexing
- Easier authorization

---

8. Record Lifecycle Management

Instead of soft delete with a `deleted_at` column, the system uses **status-based lifecycle management**:

- `students` → `status` enum (`ACTIVE` , `INACTIVE` , `TRANSFERRED` , `GRADUATED`)
- `users` → `is_active` boolean
- `notices` → `is_published` boolean + `publish_at` / `expires_at`
- `classes` , `sections` , `subjects` → `is_active` boolean

This approach provides more expressive states than a single `deleted_at` timestamp and avoids broken foreign key references.

Do **not** use soft delete for pure transactional or configuration tables unless there is a business requirement.

---

9. Password Storage

Never store plain text passwords.

Always store:

```
password_hash
```

Use:

- bcrypt (Acceptable)

---

10. ENUM Usage

Use PostgreSQL ENUM types only when values are fixed and unlikely to change.

Examples:

- Gender
- Attendance Status
- Application Status
- Notification Status

Avoid ENUMs for data that schools may customize.

---

11. Constraints

Apply database constraints wherever possible instead of relying only on backend validation.

Examples:

- UNIQUE
- FOREIGN KEY
- CHECK
- NOT NULL

The database should protect data integrity.

---

12. Indexing

Create indexes for:

- Foreign Keys
- Frequently searched fields
- UNIQUE columns

Do not create unnecessary indexes.

---

13. Audit Logging

Only business-critical actions should be stored in `audit_logs`.

Examples:

- Admission Approved
- Student Deleted
- Result Published
- Permission Changed
- Tenant Suspended

Routine reads and simple queries should not be logged.

---

14. Business Logic

Business rules should primarily live in the backend.

The database should focus on:

- Data integrity
- Relationships
- Constraints
- Performance

Avoid embedding complex business logic inside the database unless necessary.

15. Database Philosophy

The database should be:

- Fully normalized (3NF)
- Easy to understand
- Easy to extend
- Performance-oriented
- Business-driven
- Future-proof without overengineering

Table Design Template

Every table in the system will follow the same documentation structure.

1. Table Name

```
table_name
```

2. Purpose

Describe the business responsibility of the table.

3. Schema

Column	Type	Required	Default	Description

4. Primary Key

Specify the primary key.

5. Foreign Keys

List all foreign key relationships.

---

6. Unique Constraints

List all UNIQUE constraints.

---

7. Indexes

List recommended indexes.

---

8. Relationships

Describe how this table relates to other tables.

---

9. Business Rules

Document important business rules enforced by the application.

---

10. Notes

Future considerations or implementation notes.

Table Definitions

📁 Platform

Table 1 — platform_admins

Purpose

Stores Platform (Super) Administrator accounts responsible for managing the Sirius Skool platform.

Platform Admins can:

- Login to `admin.sirius-skool.com`
- Create tenants (schools)
- Suspend or activate tenants
- Assign modules
- Manage the SaaS platform

> **MVP Decision:**
> The system supports only **one seeded Platform Admin**.
> Authentication is intentionally kept minimal for simplicity.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
email	VARCHAR(255)	✅	—	Unique login email
password_hash	TEXT	✅	—	Securely hashed password (bcrypt)
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

None.

Unique Constraints

```
email
```

Only one Platform Admin account can use a given email address.

Recommended Indexes

- UNIQUE(email)

Relationships

None.

`platform_admins` is intentionally isolated from tenant data.

Business Rules

- Only one Platform Admin account exists during MVP.
- The account is seeded during initial deployment.
- Passwords must always be stored as hashes.
- Platform Admin authentication is completely separate from tenant users.
- Platform Admins never belong to any tenant.

Notes

Future versions may introduce:

- Multiple Platform Admins
- Roles & permissions
- Two-Factor Authentication (2FA)
- Last login tracking
- Profile information (name, avatar)
- Password reset workflow

These can be added without changing the existing table structure.

📁 Core & Multi-Tenant

Table 2 — tenants

Purpose

Stores the identity of each school (tenant) using the Sirius Skool platform.

A tenant represents a **single school** in the multi-tenant architecture. It is the root entity for all tenant-owned data such as users, students, attendance, results, notices, and settings.

This table is responsible only for identifying and managing the lifecycle of a school. It **does not** store authentication credentials, school configuration, or business data.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
name	VARCHAR(255)	✅	—	Official school name
subdomain	VARCHAR(100)	✅	—	Unique subdomain used for tenant identification (e.g., `abc` → `abc.sirius-skool.com`)
is_active	BOOLEAN	✅	`TRUE`	Indicates whether the tenant is active and allowed to access the system
sms_balance	INTEGER	✅	`0`	Current SMS credits available for the tenant
current_student_sequence	INTEGER	✅	`0`	Stores the last generated student sequence number for registration number generation
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

None.

This is the root table of the tenant domain.

Unique Constraints

```
subdomain
```

Every tenant must have a unique subdomain.

Recommended Indexes

- UNIQUE(subdomain)
- INDEX(is_active)

Relationships

```
tenants (1)
│
├── tenant_settings
├── tenant_modules
├── users
├── academic_years
├── classes
├── sections
├── subjects
├── applications
├── students
├── student_enrollments
├── attendance_sessions
├── attendance_records
├── exams
├── exam_subjects
├── marks
├── grade_scales
├── notices
├── notifications
└── audit_logs
```

A single tenant owns all tenant-specific data.

Business Rules

- Every school registered on the platform must have exactly one tenant record.
- The `subdomain` must be globally unique across the platform.
- A tenant can only access the platform through its assigned subdomain.
- Setting `is_active = false` immediately blocks login and access for all users of that tenant.
- Tenant authentication is performed using the subdomain and the `users` table.
- The `tenants` table must **never** store School Admin credentials or authentication data.

- Deleting a tenant is a business process handled by the application, not by deleting the database record directly.

SMS Credits

- Platform Admin allocates SMS credits to a tenant.
- School Admin and Managers consume SMS credits when sending notifications.
- Platform Admin can increase or decrease the available SMS balance at any time.
- SMS sending should always verify that sufficient `sms_balance` exists before sending.
- Each successfully sent SMS deducts the corresponding number of credits from `sms_balance`.

Student Registration Sequence

- `current_student_sequence` stores the latest sequence number used for generating student registration numbers.
- Registration numbers are generated in the format:

```
YY + Sequence
```

Example:

```
 26000001
26000002
26000003
```

- The sequence is **never reset**.
- The sequence increases continuously throughout the lifetime of the school.
- Student registration numbers are permanent and never change.
- During student admission, the backend must generate the next sequence inside a **database transaction** using row-level locking (`SELECT ... FOR UPDATE`) to prevent duplicate registration numbers under concurrent requests.

Notes

Why no email or password?

A tenant (school) is **not** a user.

Authentication belongs to the **School Admin**, whose account is stored in the `users` table.

During tenant creation, the Platform Admin provides:

- School information → stored in `tenants`
- Initial School Admin information → stored in `users`

Both records are created within a single database transaction.

Why no logo, address, phone, or branding?

These are configurable settings and belong in the `tenant_settings` table.

Keeping `tenants` focused only on tenant identity makes the database easier to maintain and extend.

Why store `current_student_sequence` here instead of a separate table?

For the MVP, the system has only one sequence that needs to be maintained: the **Student Registration Number**.

Keeping the sequence directly in the `tenants` table:

- Simplifies the database design.
- Eliminates an unnecessary table.
- Reduces joins.
- Still provides full protection against race conditions when updated inside a database transaction.

If future versions require multiple numbering systems (e.g., Invoice Number, TC Number, Receipt Number, Employee ID), a dedicated `tenant_sequences` table can be introduced without affecting the rest of the architecture.

Future Expansion

The table can be safely extended later with fields such as:

- custom_domain
- subscription_plan_id
- trial_ends_at
- suspended_at
- billing_customer_id

without affecting the existing database structure.

Table 3 — tenant_settings

Purpose

Stores configurable settings and branding information for each school (tenant).

This table contains school-specific configuration that is **not part of the tenant's identity**, such as contact information, branding, and report-related settings.

Keeping these settings separate from the `tenants` table follows the **Single Responsibility Principle (SRP)** and makes future expansion much easier.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant (school)
logo_url	TEXT	❌	`NULL`	School logo URL
address	TEXT	❌	`NULL`	Official school address
phone	VARCHAR(20)	❌	`NULL`	Official school contact number
email	VARCHAR(255)	❌	`NULL`	Official school email address
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)

Unique Constraints

```
tenant_id
```

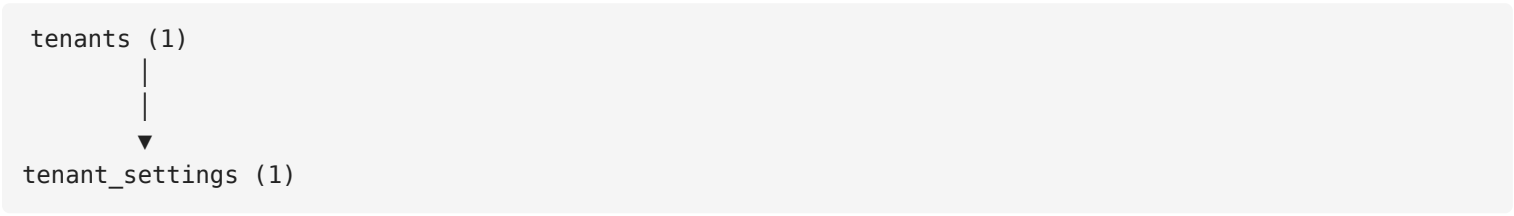
Each tenant can have only **one** settings record.

This creates a **One-to-One (1:1)** relationship between `tenants` and `tenant_settings`.

Recommended Indexes

- UNIQUE(tenant_id)

Relationships



Each school owns exactly one settings record.

Business Rules

- Every tenant should have exactly one settings record.
- School branding and contact information are stored here instead of the `tenants` table.
- Updating settings must not affect the tenant's identity.
- Authentication credentials must never be stored in this table.
- School name is stored in the `tenants` table and must not be duplicated here.

Notes

Why is the school name not stored here?

The school name identifies the tenant itself, so it belongs in the `tenants` table.

Duplicating it here would violate normalization and create unnecessary redundancy.

Why isn't the current academic year stored here?

The current academic year is **operational data**, not configuration.

It should be determined from the `academic_years` table rather than stored as a setting.

Why are SMS and Email provider settings not stored here?

The MVP only requires:

- School contact email
- School contact phone

Provider-specific credentials (SMTP, SMS Gateway, API Keys, etc.) can be introduced later without changing the current architecture.

Why aren't report templates or signatures included?

The MVP supports generating reports using the school's branding (logo and contact information).

Features such as:

- Principal Signature
- School Seal
- Custom Report Templates
- Watermarks

are intentionally deferred to future versions to keep the initial architecture simple.

Future Expansion

This table can be safely extended later with fields such as:

- website
- principal_name
- principal_signature_url
- school_seal_url
- report_header
- report_footer
- theme_color
- timezone
- language
- smtp_configuration
- sms_provider_configuration

without affecting the existing database structure.

Table 4 — tenant_modules

Purpose

Stores the modules that are available for each tenant (school).

This table acts as the feature licensing layer of the system. It determines which modules a school can access. If a module is disabled, no user (School Admin or Manager) within that tenant can access it.

The Platform Admin is responsible for enabling or disabling modules for each tenant.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant (school)
module	MODULE_ENUM	✅	—	Module assigned to the tenant
is_enabled	BOOLEAN	✅	`TRUE`	Indicates whether the module is currently enabled for the tenant
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)

Unique Constraints

```
UNIQUE (tenant_id, module)
```

A tenant can only have one record for each module.

Recommended Indexes

- INDEX(tenant_id)
- UNIQUE(tenant_id, module)

Relationships

```
 tenants (1)
      │
      ▼
tenant_modules (Many)
```

Each tenant owns multiple module configurations.

Module ENUM

```
 STUDENT_MANAGEMENT
ATTENDANCE_MANAGEMENT
RESULT_MANAGEMENT
NOTICE_BOARD
REPORTS
NOTIFICATIONS
```

> New modules can be added later without affecting the database structure.

Business Rules

- Platform Admin controls which modules are enabled for each tenant.
- A disabled module is completely inaccessible to all users within that tenant.
- School Admins cannot create or remove module assignments.
- Manager permissions are only valid if the corresponding module is enabled.
- Every newly created tenant should receive the default module set during the tenant creation process.

Notes

Why use an ENUM instead of plain text?

Using a PostgreSQL ENUM:

- Prevents spelling mistakes
- Simplifies backend validation
- Improves consistency
- Makes the ERD easier to understand

Why only one `is_enabled` field?

The MVP does not require separate states such as:

- Allocated by Platform
- Enabled by School

A single boolean is sufficient because the Platform Admin is the sole authority over module availability.

If `is_enabled = false`, the module is unavailable regardless of user permissions.

Relationship with `manager_permissions`

Manager permissions should reference a specific `tenant_modules` record instead of storing the module name again.

Example:

```
tenant_modules
      │
      ▼
manager_permissions
```

This ensures:

- No duplicated module names
- Strong referential integrity
- Permissions cannot exist for disabled or unavailable modules
- Cleaner and more maintainable authorization logic

Future Expansion

The table can later support additional licensing features such as:

- expires_at
- activated_at
- licensed_by
- subscription_plan

- trial_module
- usage_limit

without requiring any structural redesign.

📁 Authentication & Authorization

Table 5 — users

Purpose

Stores all tenant users who can authenticate and access the system.

In the MVP, a tenant has only two user roles:

- School Admin
- Manager

The `users` table is responsible only for authentication and user identity. It does **not** store module permissions, academic data, or business-specific information.

Students, parents, and teachers do **not** have login accounts in the MVP.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant (school)
full_name	VARCHAR(255)	✅	—	User's full name
email	VARCHAR(255)	✅	—	Login email address
phone	VARCHAR(20)	❌	`NULL`	Contact phone number
password_hash	TEXT	✅	—	Securely hashed password (bcrypt or Argon2id)
role	USER_ROLE	✅	—	User role within the tenant
is_active	BOOLEAN	✅	`TRUE`	Indicates whether the account is active
last_login_at	TIMESTAMPTZ	❌	`NULL`	Last successful login timestamp
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)

Unique Constraints

```
UNIQUE(email)
```

Every user email must be globally unique across the platform.

Recommended Indexes

- UNIQUE(email)
- INDEX(tenant_id)
- INDEX(role)
- INDEX(is_active)

Relationships

```
tenants (1)
      │
      ▼
users (Many)
      │
      └──────────────► manager_permissions (One-to-One or One-to-Many)
```

Each tenant owns multiple users.

A Manager's permissions are stored separately in the `manager_permissions` table.

USER_ROLE ENUM

```
 SCHOOL_ADMIN
MANAGER
```

> Platform Admins are stored separately in the `platform_admins` table.

Business Rules

- Every user belongs to exactly one tenant.
- Every user must have a unique email address.
- Users authenticate using **email and password**.
- Passwords must always be stored as secure hashes.
- A tenant can have **only one active School Admin**.
- A tenant can have multiple Managers.
- Setting `is_active = false` immediately prevents the user from logging in.
- Authentication is scoped by the tenant's subdomain and the user's email.
- Managers receive their module permissions through the `manager_permissions` table.

Authentication Flow

```
 User visits:

school.sirius-skool.com

        |
        ▼

Tenant identified from subdomain

        |
        ▼

Find tenant

        |
        ▼

Find user
WHERE tenant_id = ?
AND email = ?

        |
        ▼
```

```
Verify password hash

        |
        ▼

Generate JWT / Session
```

Notes

Why use `full_name` instead of `first_name` and `last_name`?

The platform is primarily designed for Bangladeshi schools, where names often don't fit neatly into first and last name fields.

Using a single `full_name` field:

- Simplifies forms
- Simplifies searching
- Avoids name-splitting issues
- Produces cleaner reports

Why is `phone` not used for login?

To keep authentication simple and consistent, the MVP supports **email and password** authentication only.

Phone numbers are stored solely as contact information.

Why aren't permissions stored here?

User permissions are part of the authorization system, not the authentication system.

Keeping permissions in the dedicated `manager_permissions` table provides:

- Better normalization
- Cleaner authorization logic
- Easier maintenance
- Greater flexibility for future enhancements

Why use `is_active` instead of deleting users?

Disabling a user preserves:

- Audit logs
- Attendance history
- Result publishing history
- Other historical records

This prevents broken references and maintains data integrity.

Future Expansion

The table can be safely extended later with fields such as:

- profile_photo_url
- password_changed_at
- failed_login_attempts
- account_locked_until
- two_factor_enabled
- two_factor_secret
- email_verified_at
- phone_verified_at

without requiring any structural redesign.

Table 6 — manager_permissions

Purpose

Stores module-level action permissions for Managers within a tenant.

This table implements the authorization layer of the system by defining what actions a Manager can perform in each assigned module.

Only **Managers** have records in this table. School Admins always have full access to all enabled modules and therefore do not require permission records.

This table works together with `tenant_modules` to ensure that Managers can only access modules that are enabled for their tenant.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key

Column	Type	Required	Default	Description
user_id	UUID	✅	—	References the Manager
tenant_module_id	UUID	✅	—	References the tenant's enabled module
can_view	BOOLEAN	✅	`FALSE`	Allows viewing records
can_create	BOOLEAN	✅	`FALSE`	Allows creating new records
can_edit	BOOLEAN	✅	`FALSE`	Allows updating existing records
can_delete	BOOLEAN	✅	`FALSE`	Allows deleting records
can_print	BOOLEAN	✅	`FALSE`	Allows printing reports or documents
can_export	BOOLEAN	✅	`FALSE`	Allows exporting data (PDF, Excel, CSV, etc.)
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
user_id	users(id)
tenant_module_id	tenant_modules(id)

Unique Constraints

```
UNIQUE(user_id, tenant_module_id)
```

Each Manager can have only one permission record for a specific module.

Recommended Indexes

- INDEX(user_id)
- INDEX(tenant_module_id)
- UNIQUE(user_id, tenant_module_id)

Relationships

```
 users (1)
      |
      ▼
manager_permissions (Many)
      ▲
      |
tenant_modules (1)
```

A Manager can have permissions for multiple modules.

Each permission record belongs to exactly one Manager and one tenant module.

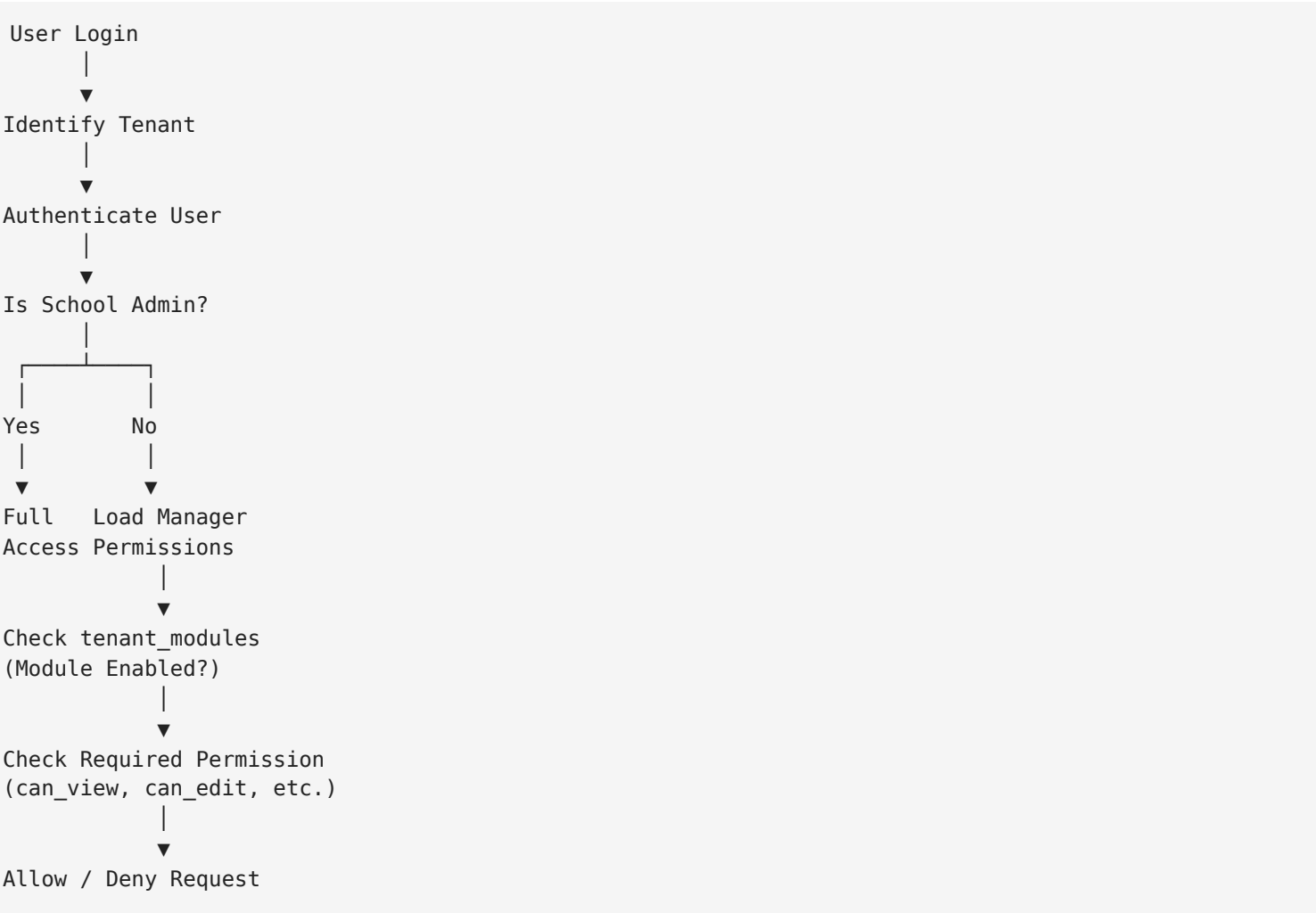
Permission Matrix

Permission	Description
can_view	View module data
can_create	Create new records
can_edit	Update existing records
can_delete	Delete records
can_print	Print reports or documents
can_export	Export data to PDF, Excel, CSV, etc.

Business Rules

- Only users with the `MANAGER` role can have permission records.
- School Admins automatically have full access to every enabled module and do not require permission records.
- A Manager can only receive permissions for modules that exist in `tenant_modules`.
- If a module is disabled in `tenant_modules`, all related permission records remain in the database but become ineffective until the module is re-enabled.
- Permission checks must always verify that:
  1. The module is enabled for the tenant.
  2. The Manager has the required permission.
- If a permission is not explicitly granted, it is considered denied.

Authorization Flow



Notes

Why reference `tenant_modules` instead of storing the module name?

Using `tenant_module_id` provides strong referential integrity.

Benefits:

- Prevents invalid module assignments.
- Eliminates duplicated module names.
- Ensures Managers can only receive permissions for modules available to their tenant.
- Simplifies future maintenance.

Why doesn't this table contain `tenant_id`?

Both related records already belong to the same tenant.

```
users
    └── tenant_id

tenant_modules
    └── tenant_id
```

Adding another `tenant_id` would duplicate data and violate normalization.

Why separate permissions into another table?

Keeping permissions separate from the `users` table:

- Follows the Single Responsibility Principle (SRP).
- Makes permission management cleaner.
- Allows adding new modules without modifying the `users` table.
- Makes the authorization system highly scalable.

Why keep permission records when a module is disabled?

Deleting permission records would cause administrators to recreate them when the module is enabled again.

By preserving the records:

- Previous configurations remain intact.
- Re-enabling a module immediately restores previous permissions.
- Administrative effort is reduced.
- Historical configuration is preserved.

Future Expansion

This table can be extended later with additional permissions such as:

- can_import
- can_approve
- can_publish
- can_archive
- can_restore

without requiring any structural redesign.

📁 Academic Structure

Table 7 — academic_years

Purpose

Stores the academic years for each tenant (school).

The Academic Year is the foundation for organizing historical academic data. Every enrollment, attendance record, examination, result, and report is associated with an academic year.

Instead of modifying historical records when students are promoted, the system creates new records for the new academic year while preserving all previous academic history.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant (school)
name	VARCHAR(50)	✅	—	Academic year name (e.g., `2026-2027`)
start_date	DATE	✅	—	Academic year start date
end_date	DATE	✅	—	Academic year end date
is_current	BOOLEAN	✅	`FALSE`	Indicates the currently active academic year
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)

Unique Constraints

```
UNIQUE (tenant_id, name)
```

A tenant cannot have two academic years with the same name.

Recommended Indexes

- INDEX(tenant_id)
- INDEX(is_current)
- UNIQUE(tenant_id, name)

Relationships

```
tenants (1)
      │
      ▼
academic_years (Many)
      │
      ├── student_enrollments
      ├── attendance_sessions
      ├── exams
      ├── notices (optional)
      └── reports
```

Each tenant owns multiple academic years.

Most academic records reference an academic year to preserve historical data.

Business Rules

- Every academic year belongs to exactly one tenant.
- A tenant can have multiple academic years.
- A tenant can have **only one** current academic year.
- Every new academic session must be created before student promotion.
- Academic years should never be deleted if they contain academic records.
- Historical academic years remain available for viewing reports, attendance, and results.

Promotion Workflow

Student promotion does **not** update historical records.

Example:

```
 Student Registration No.

26000001

Enrollment History

| Academic Year | Class | Section | Roll |
|---------------|-------|---------|------|
| 2026-2027 | Class 6 | A | 1 |
| 2027-2028 | Class 7 | B | 5 |
| 2028-2029 | Class 8 | A | 2 |

Previous attendance and results continue referencing the original academic year.
```

Notes

Why is this table necessary?

Without an Academic Year table:

- Previous results would be overwritten.
- Attendance history would become difficult to maintain.
- Student promotions would require updating historical records.
- Reports from previous years would become unreliable.

Using an Academic Year preserves the complete academic history of every student.

Why use `2026-2027` instead of only `2026`?

Different schools may follow different academic calendars.

Examples:

```
January 2026 → December 2026
```

or

```
April 2026 → March 2027
```

Using a range such as `2026-2027` works for both scenarios.

Why only `is_current`?

For the MVP, a single boolean is sufficient to identify the active academic year.

Future academic years can be created in advance while remaining `is_current = false`.

When promotion is completed:

- Previous academic year → `is_current = false`
- New academic year → `is_current = true`

Future Expansion

The table can be safely extended later with fields such as:

- status (Upcoming, Current, Closed)
- promotion_completed_at
- promoted_by
- remarks

without requiring any structural redesign.

Table 8 — classes

Purpose

Stores the list of classes available for each tenant (school).

This table defines the academic classes offered by a school (e.g., Play, Nursery, KG, Class 1, Class 2, etc.). It does **not** store which students belong to a class. Student assignments are maintained in the `student_enrollments` table.

Classes are relatively static and are reused across academic years.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant (school)
code	VARCHAR(20)	✅	—	Short unique class code (e.g., `1`, `2`, `KG`, `NUR`)
name	VARCHAR(100)	✅	—	Display name (e.g., `Class 1`, `Kindergarten`)

Column	Type	Required	Default	Description
display_order	INTEGER	✅	—	Determines the order in which classes are displayed
is_active	BOOLEAN	✅	`TRUE`	Indicates whether the class is currently available for new enrollments
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)

Unique Constraints

```
UNIQUE (tenant_id, code)

UNIQUE (tenant_id, name)
```

A tenant cannot have duplicate class codes or duplicate class names.

Recommended Indexes

- INDEX(tenant_id)
- INDEX(is_active)
- INDEX(display_order)
- UNIQUE(tenant_id, code)
- UNIQUE(tenant_id, name)

Relationships

```
 tenants (1)
      │
      ▼
classes (Many)
      │
      ├── sections
      ├── subjects
      └── student_enrollments
```

Each tenant owns multiple classes.

Classes are referenced by Sections, Subjects, and Student Enrollments.

Business Rules

- Every class belongs to exactly one tenant.
- Class definitions are shared across all academic years.
- Students are assigned to classes through the `student_enrollments` table.
- Classes should not be recreated every academic year.
- Setting `is_active = false` prevents future admissions or enrollments into that class while preserving historical data.
- Classes should not be deleted if they are referenced by existing records.

Display Order Example

Instead of sorting alphabetically:

```
 Class 1
Class 10
Class 2
```

the system uses `display_order`:

Class	Display Order
Play	1
Nursery	2
KG	3
Class 1	4
Class 2	5
Class 3	6

Class	Display Order
...	...
Class 10	13

This allows the frontend to display classes correctly.

Notes

Why isn't `academic_year_id` stored here?

Classes are permanent entities.

Students move between classes every academic year, but the class itself remains the same.

Academic history is maintained through the `student_enrollments` table, avoiding unnecessary duplication of class records.

Why have both `code` and `name`?

The `code` provides a short, stable identifier suitable for:

- Internal references
- APIs
- Integrations
- Future automation

The `name` is the human-readable value displayed throughout the application.

Example:

Code	Name
1	Class 1
2	Class 2
KG	Kindergarten
NUR	Nursery

Why use `display_order`?

Sorting by name often produces incorrect ordering (e.g., `Class 10` before `Class 2`).

Using `display_order` guarantees consistent ordering across:

- Dropdowns
- Reports
- Admission forms
- Result entry screens

Why use `is_active` instead of deleting classes?

Disabling a class preserves historical references such as:

- Student enrollments
- Attendance records
- Examination results
- Academic reports

while preventing the class from being used for future admissions.

Future Expansion

The table can be safely extended later with fields such as:

- description
- class_teacher_id
- maximum_students
- minimum_age
- maximum_age

without requiring any structural redesign.

Table 9 — sections

Purpose

Stores the sections available under each class for a school.

A section represents a subdivision of a class (e.g., Class 6 → A, B, C). Sections are permanent academic entities and are reused across academic years. Students are assigned to sections through the `student_enrollments` table.

Each section belongs to **exactly one class**.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant
class_id	UUID	✅	—	References the parent class
code	VARCHAR(20)	✅	—	Short section code (e.g., `A`, `B`, `C`)
name	VARCHAR(100)	✅	—	Display name (e.g., `Section A`)
display_order	INTEGER	✅	—	Determines the display order of sections
is_active	BOOLEAN	✅	`TRUE`	Indicates whether the section is available for new enrollments
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)
class_id	classes(id)

Unique Constraints

```
UNIQUE(class_id, code)

UNIQUE(class_id, name)
```

A class cannot contain two sections with the same code or name.

Recommended Indexes

- INDEX(tenant_id)
- INDEX(class_id)
- INDEX(is_active)
- INDEX(display_order)
- UNIQUE(class_id, code)
- UNIQUE(class_id, name)

Relationships

```
tenants (1)
      │
      ▼
classes (1)
      │
      ▼
sections (Many)
      │
      └── student_enrollments
```

Each class owns multiple sections.

Sections are referenced by Student Enrollments to determine where a student studies during an academic year.

Business Rules

- Every section belongs to exactly one class.
- Sections are shared across all academic years.
- Students are assigned to sections through the `student_enrollments` table.
- A class may contain multiple sections.
- Section codes only need to be unique within the same class.
- Setting `is_active = false` prevents future enrollments while preserving historical records.
- Sections should not be deleted if they are referenced by existing records.

Example Structure

```
Class 6
├── Section A
├── Section B
└── Section C

Class 7
```

```
├── Section A
└── Section B
```

Notice that:

- Class 6 → Section A
- Class 7 → Section A

are two different sections and are both valid.

Notes

Why doesn't this table contain `academic_year_id`?

Sections are permanent academic structures.

Students change sections over the years, but the section definitions remain the same.

Academic history is maintained through the `student_enrollments` table rather than duplicating section records every year.

Why have both `code` and `name`?

The `code` is a short internal identifier suitable for:

- APIs
- Internal references
- Integrations

The `name` is displayed throughout the application.

Example:

Code	Name
A	Section A
B	Section B
C	Section C

Why use `display_order`?

Sorting alphabetically is not always sufficient.

Using `display_order` ensures consistent ordering across:

- Admission forms
- Attendance pages
- Result entry
- Reports
- Dropdown menus

Why use `is_active` instead of deleting sections?

Disabling a section preserves historical references such as:

- Student enrollments
- Attendance records
- Examination results
- Academic reports

while preventing the section from being used for future admissions.

Why store both `class_id` and `section_id` in `student_enrollments`?

Although a section already belongs to a class, storing both values in `student_enrollments` is an intentional optimization.

Benefits:

- Faster queries
- Simpler filtering
- Fewer joins
- Better reporting performance

The application must validate that the selected `section_id` belongs to the selected `class_id`.

Future Expansion

The table can be safely extended later with fields such as:

- room_number
- floor
- maximum_students
- homeroom_manager_id
- remarks

without requiring any structural redesign.

Table 10 — subjects

Purpose

Stores the subjects offered for each class within a tenant.

A subject represents a course that students study in a specific class (e.g., Mathematics, English, Physics). Subjects belong to classes because different classes may have different curricula.

This table defines the curriculum structure only. It **does not** store exam-specific information such as full marks, pass marks, or grading rules. Those are maintained in the `exam_subjects` table to support flexible examination structures.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant
class_id	UUID	✅	—	References the class
code	VARCHAR(30)	✅	—	Short subject code (e.g., `MATH`, `ENG`, `PHY`)
name	VARCHAR(100)	✅	—	Subject name (e.g., `Mathematics`)
subject_type	SUBJECT_TYPE	✅	`MANDATORY`	Indicates whether the subject is mandatory or optional
display_order	INTEGER	✅	—	Determines the display order of subjects
is_active	BOOLEAN	✅	`TRUE`	Indicates whether the subject is currently available
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)
class_id	classes(id)

Unique Constraints

```
 UNIQUE(class_id, code)

UNIQUE(class_id, name)
```

A class cannot contain duplicate subject codes or duplicate subject names.

Recommended Indexes

- INDEX(tenant_id)
- INDEX(class_id)
- INDEX(subject_type)
- INDEX(is_active)
- INDEX(display_order)
- UNIQUE(class_id, code)
- UNIQUE(class_id, name)

Relationships

```
 tenants (1)
      │
      ▼
classes (1)
      │
      ▼
subjects (Many)
      │
      ▼
exam_subjects
      │
      ▼
marks
```

Each class owns multiple subjects.

Exam-specific configurations such as marking rules are defined in the `exam_subjects` table.

SUBJECT_TYPE ENUM

```
MANDATORY

OPTIONAL
```

Business Rules

- Every subject belongs to exactly one class.
- Different classes may have subjects with the same name.
- Subject definitions are shared across all academic years.
- A class cannot contain duplicate subject names or codes.
- Subject marking rules must **not** be stored in this table.
- Exam-specific configurations are maintained in the `exam_subjects` table.
- Setting `is_active = false` prevents the subject from being assigned to new exams while preserving historical records.
- Subjects should not be deleted if they are referenced by examinations or results.

Example Structure

```
Class 6
├── Mathematics
├── English
├── Science
└── ICT

Class 9
├── Mathematics
├── Higher Mathematics
├── Physics
├── Chemistry
└── Biology
```

Each class maintains its own curriculum independently.

Notes

Why do subjects belong to classes?

Different classes often have different curricula.

For example:

- Class 1 may offer Mathematics and English.
- Class 9 may additionally offer Physics, Chemistry, and Higher Mathematics.

Associating subjects directly with classes keeps the curriculum simple and flexible.

Why aren't `full_marks` and `pass_marks` stored here?

Different examinations may use different marking schemes.

Example:

Examination	Mathematics
Mid-Term	50 Marks
Final	100 Marks

If marks were stored in this table, supporting different exam structures would become difficult.

Instead, exam-specific configurations are stored in the `exam_subjects` table.

Why have both `code` and `name`?

The `code` provides a short, stable identifier suitable for:

- APIs
- Integrations
- Internal references

The `name` is displayed throughout the application.

Example:

Code	Name
MATH	Mathematics
ENG	English
PHY	Physics

Why use `subject_type`?

Some schools offer optional subjects in addition to mandatory ones.

Examples include:

- Higher Mathematics
- Agriculture
- ICT (depending on curriculum)

Using `subject_type` allows the system to support optional subjects without requiring structural changes.

Why use `is_active` instead of deleting subjects?

Disabling a subject preserves historical references such as:

- Examination records
- Student results
- Report cards
- Academic reports

while preventing it from being assigned to future examinations.

Future Expansion

The table can be safely extended later with fields such as:

- description
- syllabus
- credit_hours
- laboratory_required
- practical_required

without requiring any structural redesign.

Table 11 — class_subjects

Purpose

Junction table linking subjects to classes. This functionality has been **merged into the `subjects` table** via `class_id`.

Status

Merged / Deprecated — The `subjects` table already contains a `class_id` column, which directly assigns each subject to its parent class. No separate `class_subjects` junction table is needed in the current schema. This entry is retained for completeness and reference only.

📁 Student Management

Table 12 — applications

Purpose

Stores admission applications submitted by prospective students.

An application represents a student's request for admission. Applicants are **not** considered students until their application is approved.

This table manages the complete admission workflow, including submission, review, approval, and rejection.

Once approved, the system creates records in the `students` and `student_enrollments` tables while preserving the original application for historical reference.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant
application_no	VARCHAR(30)	✅	—	Human-readable application number (e.g., `APP-2026-000001`)
academic_year_id	UUID	✅	—	Academic year the applicant is applying for
applying_class_id	UUID	✅	—	Requested class for admission
full_name	VARCHAR(255)	✅	—	Applicant's full name
gender	GENDER	✅	—	Applicant's gender
date_of_birth	DATE	✅	—	Applicant's date of birth
guardian_name	VARCHAR(150)	✅	—	Parent or guardian's name
guardian_phone	VARCHAR(20)	✅	—	Parent or guardian's phone number
address	TEXT	✅	—	Applicant's address
status	APPLICATION_STATUS	✅	`PENDING`	Current application status
reviewed_by	UUID	❌	`NULL`	User who reviewed the application

Column	Type	Required	Default	Description
reviewed_at	TIMESTAMPTZ	❌	`NULL`	Date and time of review
rejection_reason	TEXT	❌	`NULL`	Reason for rejection (if applicable)
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)
academic_year_id	academic_years(id)
applying_class_id	classes(id)
reviewed_by	users(id)

Unique Constraints

```
UNIQUE(tenant_id, application_no)
```

Each application number must be unique within a tenant.

Recommended Indexes

- INDEX(tenant_id)
- INDEX(academic_year_id)
- INDEX(applying_class_id)
- INDEX(status)
- INDEX(created_at)
- UNIQUE(tenant_id, application_no)

Relationships

```
 tenants (1)
       │
       ▼
applications (Many)
       │
       ├── academic_years
       ├── classes
       └── users (reviewed_by)

Upon Approval
       │
       ├── students
       └── student_enrollments
```

Applications are the entry point of the admission process.

Approved applications create official student records.

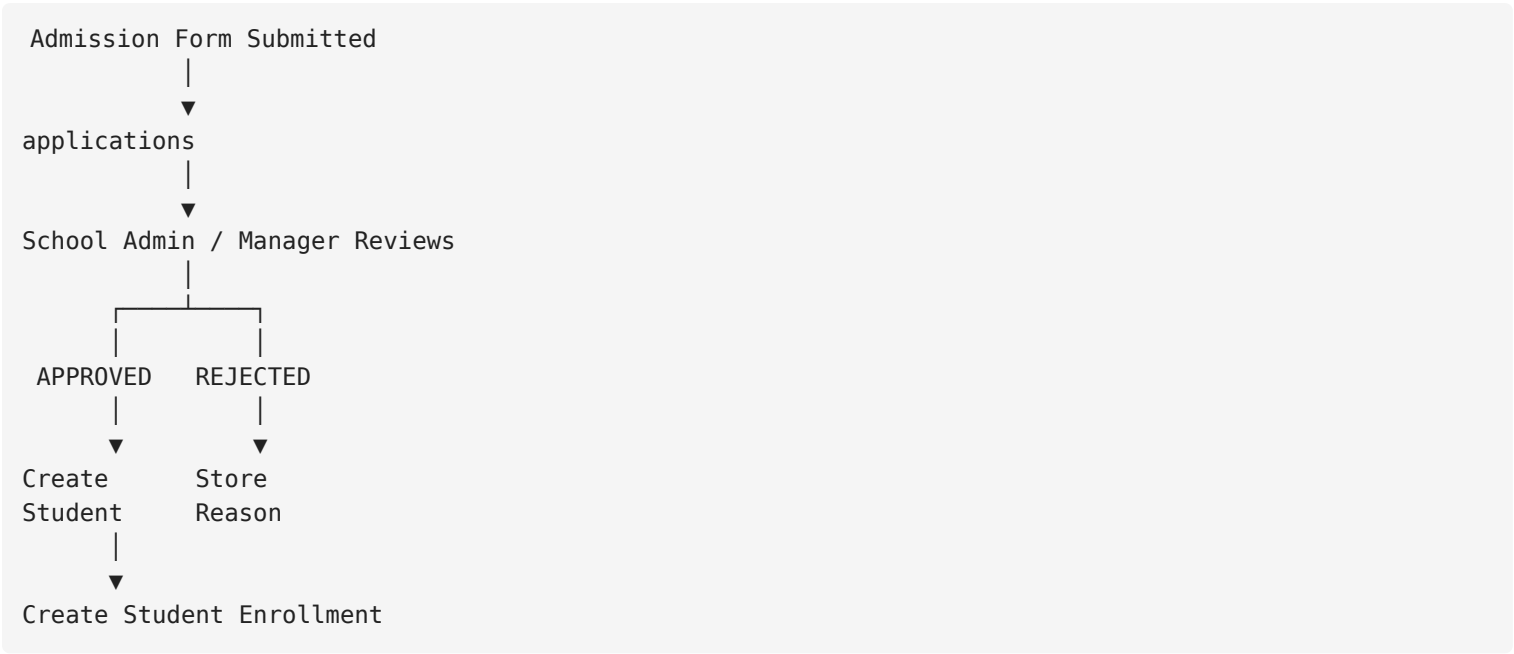
APPLICATION_STATUS ENUM

```
 PENDING

APPROVED

REJECTED
```

Business Rules

- Every application belongs to exactly one tenant.
- Every application must target one academic year.
- Every application must specify one class.
- Applicants are **not** students until their application is approved.
- Approving an application creates:
  - One record in `students`
  - One record in `student_enrollments`
- Rejecting an application does **not** create a student.
- Approved or rejected applications must remain in the database for historical purposes.
- Once reviewed, `reviewed_by` and `reviewed_at` should be recorded.
- `rejection_reason` is required when the status is `REJECTED`.

Admission Workflow



Notes

Why have a separate `applications` table?

Applicants and students are different business entities.

Keeping them separate:

- Prevents rejected applicants from becoming students.
- Supports an approval workflow.
- Preserves the original admission form.
- Keeps the `students` table clean and accurate.

Why create a new student record after approval?

The application represents the information submitted by the applicant.

The student record represents the school's official academic record.

This separation allows student information (such as address or guardian phone number) to be updated later without altering the original application.

Why not assign a section during admission?

At the time of admission, the school may not yet know which section the student will join.

Section assignment occurs during enrollment after admission approval.

Therefore, this table stores only the requested class.

Why use an application number?

A human-readable application number is easier for school staff to reference than a UUID.

Example:

```
APP-2026-000001
```

instead of

```
9d4f9b2a-2cb6-4a1b...
```

Future Expansion

The table can be safely extended later with fields such as:

- applicant_photo
- previous_school
- previous_result
- birth_certificate_no
- admission_test_score
- admission_notes
- documents_verified

without requiring any structural redesign.

Table 13 — students

Purpose

Stores the permanent master profile of every admitted student.

A student record is created only after an admission application is approved. Unlike the `applications` table, which is temporary, this table serves as the permanent source of truth for all student information throughout the student's academic life.

This table stores only **permanent personal information**. Academic information such as class, section, roll number, and academic year is maintained in the `student_enrollments` table.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant
registration_no	VARCHAR(20)	✅	—	Permanent registration number (e.g., `26000001`)
full_name	VARCHAR(255)	✅	—	Student's full name
gender	GENDER	✅	—	Student's gender
date_of_birth	DATE	✅	—	Student's date of birth
guardian_name	VARCHAR(150)	✅	—	Parent or guardian's name
guardian_phone	VARCHAR(20)	✅	—	Parent or guardian's phone number
address	TEXT	✅	—	Student's residential address
admission_date	DATE	✅	—	Official admission date
status	STUDENT_STATUS	✅	`ACTIVE`	Current student status
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)

Unique Constraints

```
UNIQUE(tenant_id, registration_no)
```

Each registration number must be unique within a tenant.

Recommended Indexes

- INDEX(tenant_id)
- INDEX(registration_no)
- INDEX(status)
- UNIQUE(tenant_id, registration_no)

Relationships

```
tenants (1)
      │
      ▼
students (Many)
      │
      ├── student_enrollments
      ├── attendance_records
      ├── marks
      ├── notifications
      └── audit_logs
```

The `students` table is the central entity for all student-related operations.

STUDENT_STATUS ENUM

```
ACTIVE

INACTIVE

TRANSFERRED

GRADUATED
```

Business Rules

- Every student belongs to exactly one tenant.
- A student is created only after an admission application is approved.
- Every student receives a permanent registration number.
- The registration number never changes throughout the student's lifetime.

- Personal information may be updated without affecting academic history.
- Class, section, roll number, and academic year must **not** be stored in this table.
- Student promotions must never update this table.
- Setting `status` to `TRANSFERRED` or `GRADUATED` prevents the student from participating in future academic activities while preserving historical records.
- Admission applications may be permanently deleted after the configured retention period without affecting this table.

Registration Number Example

```
 26000001
26000002
26000003
```

The registration number is permanent and remains unchanged regardless of promotions.

Example:

Academic Year	Class	Registration No
2026-2027	Class 6	26000001
2027-2028	Class 7	26000001
2028-2029	Class 8	26000001

Notes

Why doesn't this table store class, section, or roll number?

These values change every academic year.

Storing them here would require updating the student record during every promotion, causing loss of historical academic information.

Instead, they are maintained in the `student_enrollments` table.

Why isn't `application_id` stored?

Admission applications are temporary records and may be permanently deleted after a defined retention period (e.g., 90 days).

To avoid broken foreign key references, the student record is created independently by copying the required information during admission approval.

Once created, the `students` table becomes the permanent source of truth.

Why use `status` instead of deleting students?

Historical data such as:

- Attendance
- Examination results
- Report cards
- Audit logs

must remain available even after a student graduates or transfers.

Changing the student's status preserves all historical records while preventing future academic operations.

Future Expansion

The table can be safely extended later with fields such as:

- photo_url
- blood_group
- religion
- nationality
- birth_certificate_no
- emergency_contact_name
- emergency_contact_phone
- remarks

without requiring any structural redesign.

Table 14 — student_enrollments

Purpose

Stores a student's academic enrollment history.

A new enrollment record is created whenever a student is admitted or promoted to a new academic year. This table tracks the student's academic journey, including class, section, roll number, and enrollment status, while preserving complete historical records.

Unlike the `students` table, which stores permanent personal information, this table stores information that changes over time.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant
student_id	UUID	✅	—	References the student
academic_year_id	UUID	✅	—	References the academic year
class_id	UUID	✅	—	References the class
section_id	UUID	✅	—	References the section
roll_number	INTEGER	✅	—	Roll number within the section for the academic year
enrollment_date	DATE	✅	—	Date the student was admitted or promoted into this enrollment
status	ENROLLMENT_STATUS	✅	`ACTIVE`	Current enrollment status
previous_enrollment_id	UUID	❌	`NULL`	References the student's previous enrollment record
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)
student_id	students(id)
academic_year_id	academic_years(id)
class_id	classes(id)
section_id	sections(id)

Column	References
previous_enrollment_id	student_enrollments(id)

Unique Constraints

```
 UNIQUE(student_id, academic_year_id)

UNIQUE(academic_year_id, section_id, roll_number)
```

Rules:

- A student can only have one enrollment in a specific academic year.
- A roll number must be unique within the same section and academic year.

Recommended Indexes

- INDEX(tenant_id)
- INDEX(student_id)
- INDEX(academic_year_id)
- INDEX(class_id)
- INDEX(section_id)
- INDEX(status)
- INDEX(previous_enrollment_id)
- UNIQUE(student_id, academic_year_id)
- UNIQUE(academic_year_id, section_id, roll_number)

Relationships

```
 students (1)
      │
      ▼
student_enrollments (Many)
      │
      ├── attendance_records
      ├── exams
      ├── marks
      └── reports
```

Each student can have multiple enrollment records throughout their academic life.

Academic operations always reference the appropriate enrollment record rather than the student directly.

ENROLLMENT_STATUS ENUM

```
 ACTIVE

PROMOTED

TRANSFERRED

GRADUATED
```

Business Rules

- Every enrollment belongs to exactly one student.
- Every enrollment belongs to one academic year.
- Every enrollment belongs to one class and one section.
- Roll numbers restart for each section in every academic year.
- A student cannot have multiple enrollments in the same academic year.
- Promotion creates a **new** enrollment record instead of modifying the existing one.
- Previous enrollment records must never be deleted or overwritten.
- The application must validate that the selected section belongs to the selected class.
- `previous_enrollment_id` should reference the student's immediately preceding enrollment when applicable.

Promotion Workflow

```
 Student
      |
      ▼
2026-2027
Class 6
Section A
Roll 1
Status: PROMOTED
      |
      ▼
2027-2028
Class 7
Section B
Roll 5
Status: ACTIVE
```

Promotion creates a new enrollment while preserving the previous academic history.

Roll Number Example

```
 Academic Year: 2026-2027

Class 6
└── Section A
    ├── Roll 1
    ├── Roll 2
    ├── Roll 3

Class 6
└── Section B
    ├── Roll 1
    ├── Roll 2
```

Roll numbers are unique only within the same section and academic year.

Notes

Why doesn't the `students` table store class or roll number?

A student's class, section, and roll number change over time.

Keeping them in the `students` table would require updating the same record every year, resulting in loss of academic history.

This table preserves every academic session independently.

Why store both `class_id` and `section_id`?

Although each section belongs to a class, storing both values is an intentional optimization.

Benefits:

- Faster queries
- Simpler reporting
- Reduced joins
- Better dashboard performance

The application must always verify that the selected section belongs to the selected class.

Why use `previous_enrollment_id`?

This creates a linked academic history.

Benefits include:

- Easy promotion tracking
- Academic timeline generation
- Transfer history
- Simpler historical navigation

Example:

```
 2026 Enrollment
        │
        ▼
2027 Enrollment
        │
        ▼
2028 Enrollment
```

Why create a new record during promotion?

Updating an existing enrollment would overwrite historical data.

Creating a new enrollment preserves:

- Previous class
- Previous section
- Previous roll number
- Previous academic year

while accurately representing the student's progression.

Future Expansion

The table can be safely extended later with fields such as:

- admission_type
- house
- shift
- group
- promoted_by
- promotion_date

- remarks

without requiring any structural redesign.

📁 Attendance

Table 15 — attendance_sessions

Purpose

Stores one attendance session for a specific class, section, and academic year on a particular date.

An attendance session acts as the parent record for all student attendance records taken on that day. It stores information about **who took the attendance**, **when it was submitted**, **its current workflow status**, and **whether absence notifications have been sent**.

This design eliminates duplicate data, supports attendance corrections, and provides a clean workflow for attendance submission and SMS notifications.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant
academic_year_id	UUID	✅	—	References the academic year
class_id	UUID	✅	—	References the class
section_id	UUID	✅	—	References the section
attendance_date	DATE	✅	—	Date of attendance
taken_by	UUID	✅	—	User who initially took the attendance
status	ATTENDANCE_SESSION_STATUS	✅	`DRAFT`	Current attendance workflow status
submitted_at	TIMESTAMPTZ	❌	`NULL`	Date and time when attendance was submitted
sms_sent_at	TIMESTAMPTZ	❌	`NULL`	Date and time when absence SMS was last sent
last_modified_by	UUID	❌	`NULL`	User who last modified the attendance session
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp

Column	Type	Required	Default	Description
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)
academic_year_id	academic_years(id)
class_id	classes(id)
section_id	sections(id)
taken_by	users(id)
last_modified_by	users(id)

Unique Constraints

```
UNIQUE(academic_year_id, section_id, attendance_date)
```

A section can have only one attendance session for a given academic year and date.

Recommended Indexes

- INDEX(tenant_id)
- INDEX(academic_year_id)
- INDEX(class_id)
- INDEX(section_id)
- INDEX(attendance_date)
- INDEX(status)
- INDEX(taken_by)
- UNIQUE(academic_year_id, section_id, attendance_date)

Relationships

```
attendance_sessions (1)
          │
          ▼
attendance_records (Many)
```

Each attendance session contains the attendance records of all students in a section for a specific date.

ATTENDANCE_SESSION_STATUS ENUM

```
DRAFT

SUBMITTED
```

Business Rules

- One attendance session represents one section on one date.
- A section cannot have multiple attendance sessions for the same date.
- Attendance begins in the `DRAFT` state.
- Teachers may update attendance while the session is in `DRAFT`.
- Once submitted, the status changes to `SUBMITTED`.
- Submitted attendance is included in reports.
- Attendance may still be corrected after submission by authorized users.
- Every modification must be recorded in the `audit_logs` table.
- `sms_sent_at` is updated whenever absence notifications are successfully sent.
- Attendance corrections do not require creating a new attendance session.

Attendance Workflow

```
Create Attendance Session
          │
          ▼
Status = DRAFT
          │
          ▼
Teacher Marks Attendance
          │
          ▼
Submit Attendance
          │
```

```
          ▼
Status = SUBMITTED
          │
          ▼
(Optional)
Send SMS to Guardians of Absent Students
          │
          ▼
sms_sent_at Updated
          │
          ▼
(Optional)
Authorized User Corrects Attendance
          │
          ▼
Audit Log Created Automatically
```

Notes

Why have a separate `attendance_sessions` table?

Without this table, information such as:

- attendance date
- class
- section
- teacher
- submission time

would have to be duplicated for every student.

For a class of 50 students, the same information would be stored 50 times.

Using a parent session eliminates redundancy and keeps the database normalized.

Why use `status` instead of `is_locked`?

Schools often need to correct attendance after submission due to:

- Late arrivals
- Teacher mistakes
- Approved leave requests
- Administrative corrections

Using a workflow status provides flexibility while relying on permissions and audit logs for accountability.

Why store `sms_sent_at`?

This field allows the system to know whether absence notifications have already been sent.

Benefits include:

- Preventing accidental duplicate SMS
- Showing SMS status in the attendance screen
- Allowing administrators to resend notifications when necessary

---

Why store `last_modified_by`?

Although every change is recorded in `audit_logs`, this field makes it easy to display the most recent editor without querying audit history.

Future Expansion

The table can be safely extended later with fields such as:

- submission_notes
- approved_by
- approved_at
- total_present
- total_absent
- total_late
- total_leave

without requiring any structural redesign.

Table 16 — attendance_records

Purpose

Stores the attendance status of each enrolled student for a specific attendance session.

Each record represents **one student's attendance** for **one attendance session**. The actual attendance date, class, section, and teacher information are inherited from the parent `attendance_sessions` record.

This table only stores student-specific attendance information, keeping the database fully normalized and eliminating redundant data.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
attendance_session_id	UUID	✅	—	References the attendance session
student_enrollment_id	UUID	✅	—	References the student's enrollment for the academic year
attendance_status	ATTENDANCE_STATUS	✅	`PRESENT`	Student's attendance status
remarks	TEXT	❌	`NULL`	Optional remarks or explanation
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
attendance_session_id	attendance_sessions(id)
student_enrollment_id	student_enrollments(id)

Unique Constraints

```
UNIQUE(attendance_session_id, student_enrollment_id)
```

A student can have only one attendance record within a single attendance session.

Recommended Indexes

- INDEX(attendance_session_id)
- INDEX(student_enrollment_id)
- INDEX(attendance_status)

- UNIQUE(attendance_session_id, student_enrollment_id)

Relationships

```
attendance_sessions (1)
          │
          ▼
attendance_records (Many)
          │
          ▼
student_enrollments (1)
```

Each attendance session contains one attendance record for every enrolled student in that class and section.

ATTENDANCE_STATUS ENUM

```
PRESENT

ABSENT

LATE

LEAVE
```

Business Rules

- Every attendance record belongs to exactly one attendance session.
- Every attendance record belongs to exactly one student enrollment.
- A student cannot have multiple attendance records within the same attendance session.
- Attendance records inherit the academic year, class, section, and attendance date from the parent `attendance_sessions` record.
- Editing attendance records after submission is allowed only for authorized users.
- Every modification must be recorded in the `audit_logs` table.
- Deleting attendance records is not permitted. Corrections should be made by updating the existing record.

Example

```
Attendance Session
_____________________________
```

```
Date      : 10 Jul 2026
Class     : Class 6
Section   : A

Attendance Records
──────────────────────────────────
Roll 1  → PRESENT
Roll 2  → ABSENT
Roll 3  → LATE
Roll 4  → PRESENT
Roll 5  → LEAVE
```

One attendance session contains the attendance of all students for that day.

Notes

Why reference `student_enrollments` instead of `students`?

Attendance belongs to a student's **academic enrollment**, not to the permanent student profile.

Example:

```
 Student
Registration No: 26000001

2026–2027
Class 6
Section A
↓
Attendance Records

2027–2028
Class 7
Section B
↓
New Attendance Records
```

This automatically preserves attendance history across promotions.

Why doesn't this table store `tenant_id`, `class_id`, `section_id`, or `attendance_date`?

Those values already exist in the parent `attendance_sessions` table.

Storing them here would create unnecessary duplication and increase storage requirements without providing any benefit.

Why use `attendance_status` instead of `status`?

Using a descriptive column name improves readability and avoids ambiguity when joining multiple tables that also contain a `status` field.

Example:

```
attendance_status = 'ABSENT'
```

is much clearer than:

```
status = 'ABSENT'
```

Why include `remarks`?

Some attendance records require additional context.

Examples:

- Late due to traffic
- Medical leave
- Family emergency
- School event

Keeping remarks optional allows schools to document exceptional cases without affecting normal attendance processing.

Future Expansion

The table can be safely extended later with fields such as:

- arrival_time
- departure_time
- leave_type
- attachment_url
- verified_by
- verification_date

without requiring any structural redesign.

📁 Examination & Results

Table 17 — exams

Purpose

Stores examination events for a specific class and academic year.

An exam represents an assessment event (e.g., Mid-Term, Final, Monthly Test) conducted for a class. It defines **when the exam takes place**, **its current lifecycle**, and **when results are officially published**.

This table **does not store subjects or marks**. Those are managed by the `exam_subjects` and `marks` tables, allowing each examination to have its own marking structure.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant
academic_year_id	UUID	✅	—	References the academic year
class_id	UUID	✅	—	References the class
name	VARCHAR(100)	✅	—	Examination name (e.g., Mid-Term, Final Exam)
start_date	DATE	✅	—	Examination start date
end_date	DATE	✅	—	Examination end date
status	EXAM_STATUS	✅	`DRAFT`	Current examination status
created_by	UUID	✅	—	User who created the examination
result_published_at	TIMESTAMPTZ	❌	`NULL`	Date and time when results were officially published
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)
academic_year_id	academic_years(id)
class_id	classes(id)
created_by	users(id)

Unique Constraints

```
UNIQUE(academic_year_id, class_id, name)
```

A class cannot have two examinations with the same name in the same academic year.

Recommended Indexes

- INDEX(tenant_id)
- INDEX(academic_year_id)
- INDEX(class_id)
- INDEX(status)
- INDEX(start_date)
- INDEX(end_date)
- UNIQUE(academic_year_id, class_id, name)

Relationships

```
 academic_years (1)
        │
        ▼
classes (1)
        │
        ▼
exams (Many)
        │
        ▼
exam_subjects (Many)
        │
        ▼
marks (Many)
```

Each examination belongs to one class and one academic year and contains multiple exam subjects.

EXAM_STATUS ENUM

```
 DRAFT

PUBLISHED

COMPLETED
```

Business Rules

- Every examination belongs to exactly one tenant.
- Every examination belongs to one academic year.
- Every examination belongs to one class.
- A class cannot have duplicate examination names within the same academic year.
- Examinations begin in the `DRAFT` state.
- Subjects can be configured while the examination is in the `DRAFT` state.
- Marks entry becomes available after the examination is published.
- Results become visible to students and parents only after `result_published_at` is set.
- Completing an examination does not automatically publish results.
- Examination records should never be deleted once marks have been entered.

Examination Workflow

```
 Create Examination
        |
        ▼
Status = DRAFT
        |
        ▼
Configure Exam Subjects
        |
        ▼
Status = PUBLISHED
        |
        ▼
Teachers Enter Marks
        |
        ▼
Status = COMPLETED
        |
        ▼
Publish Results
        |
```

```
        ▼
result_published_at Recorded
```

Example

```
 Academic Year: 2026–2027

Class: 6

Exam: Final Examination

Duration:
01 Dec 2026 → 10 Dec 2026

Status:
COMPLETED

Results Published:
15 Dec 2026
```

Notes

Why doesn't this table store subjects?

An examination may contain many subjects.

Example:

```
 Final Examination
│
├── Bangla
├── English
├── Mathematics
├── Science
└── ICT
```

This one-to-many relationship is managed by the `exam_subjects` table.

Why doesn't this table store marks?

Different subjects may have different:

- Full marks
- Pass marks

- Theory marks
- Practical marks

These configurations belong to `exam_subjects`, while individual student scores belong to the `marks` table.

Why use `result_published_at`?

An examination may finish before results are published.

Example:

```
 Exam Ends
05 July

↓

Teachers Complete Marking
10 July

↓

Results Published
15 July
```

Recording the publication time allows the system to:

- Notify students and parents
- Control result visibility
- Generate publication reports
- Maintain an audit trail

Why use `status`?

The examination lifecycle is different from result publication.

Examples:

- `DRAFT` → Exam is being configured.
- `PUBLISHED` → Subjects were set and record status after.
- `COMPLETED` → Mark entry is finished.

Result publication is controlled separately through `result_published_at`.

Future Expansion

The table can be safely extended later with fields such as:

- description
- examination_type
- grading_policy_id
- published_by
- result_locked_at
- remarks

without requiring any structural redesign.

Table 18 — exam_subjects

Purpose

Stores the examination configuration for each subject within an examination.

An exam subject represents **how a specific subject is assessed in a particular examination**. It defines the marking scheme, passing criteria, and optional theory/practical mark distribution.

This table separates the academic curriculum from the examination configuration, allowing different examinations to use different marking structures for the same subject.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
exam_id	UUID	✅	—	References the examination
subject_id	UUID	✅	—	References the subject
full_marks	DECIMAL(5,2)	✅	—	Maximum obtainable marks
pass_marks	DECIMAL(5,2)	✅	—	Minimum marks required to pass
theory_marks	DECIMAL(5,2)	❌	`NULL`	Maximum theory marks (if applicable)
practical_marks	DECIMAL(5,2)	❌	`NULL`	Maximum practical marks (if applicable)
display_order	INTEGER	✅	—	Subject appearance order on reports and mark sheets
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
exam_id	exams(id)
subject_id	subjects(id)

Unique Constraints

```
UNIQUE(exam_id, subject_id)
```

A subject can appear only once within the same examination.

Recommended Indexes

- INDEX(exam_id)
- INDEX(subject_id)
- INDEX(display_order)
- UNIQUE(exam_id, subject_id)

Relationships

```
 exams (1)
      |
      ▼
exam_subjects (Many)
      |
      ▼
marks (Many)

subjects (1)
      |
      ▼
exam_subjects (Many)
```

Each examination consists of multiple exam subjects, and each exam subject stores the assessment configuration for one subject.

Business Rules

- Every exam subject belongs to exactly one examination.
- Every exam subject references exactly one subject.
- A subject cannot be added multiple times to the same examination.
- `full_marks` must be greater than zero.
- `pass_marks` must be less than or equal to `full_marks`.
- If `theory_marks` and `practical_marks` are provided, their sum must equal `full_marks`.
- If a subject has no practical component, both `theory_marks` and `practical_marks` may remain `NULL`.
- Examination-specific marking rules must be stored here, not in the `subjects` table.

Example

```
Final Examination

├── Bangla
│   ├── Full Marks: 100
│   └── Pass Marks: 33
│
├── English
│   ├── Full Marks: 100
│   └── Pass Marks: 33
│
├── Mathematics
│   ├── Full Marks: 100
│   └── Pass Marks: 40
│
└── ICT
    ├── Full Marks: 50
    └── Pass Marks: 20
```

Each subject may have different marking rules depending on the examination.

Theory & Practical Example

```
 Physics

Theory Marks     : 70
Practical Marks  : 30
--------------------------
```

```
Full Marks       : 100
Pass Marks       : 33
```

Another subject may simply have:

```
 Mathematics

Full Marks: 100
Pass Marks: 40

Theory Marks     : NULL
Practical Marks  : NULL
```

This provides flexibility without requiring separate tables.

Notes

Why not store `full_marks` in the `subjects` table?

Different examinations may have different marking schemes.

Example:

Examination	Mathematics
Mid-Term	50
Final	100
Monthly Test	25

Storing marking rules in the `subjects` table would make this impossible.

Keeping them in `exam_subjects` allows every examination to define its own assessment structure.

Why have a separate `exam_subjects` table?

This table separates:

- **Curriculum** (`subjects`)
- **Assessment Configuration** (`exam_subjects`)
- **Student Performance** (`marks`)

This normalization makes the Results module highly flexible and easy to extend.

Why include `display_order`?

Schools often want report cards to display subjects in a specific order.

Example:

```
 1. Bangla
2. English
3. Mathematics
4. Science
5. ICT
```

Using `display_order` avoids relying on alphabetical sorting.

Why are `theory_marks` and `practical_marks` optional?

Not every subject has theory and practical components.

Examples:

- Mathematics → Theory only
- ICT → Theory + Practical
- Physics → Theory + Practical
- Drawing → Practical only (future possibility)

Making these fields nullable supports different examination structures without adding unnecessary complexity.

Future Expansion

The table can be safely extended later with fields such as:

- exam_date
- start_time
- end_time
- duration_minutes
- room_number
- examiner_id
- instructions

without requiring any structural redesign.

Table 19 — marks

Purpose

Stores the raw marks obtained by each student for every subject in an examination.

Each record represents **one student's score for one exam subject**. This table stores only the source data (obtained marks and attendance in the exam). All calculated values such as grades, GPA, percentage, pass/fail, merit position, and total marks are generated dynamically using this data.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
exam_subject_id	UUID	✅	—	References the exam subject
student_enrollment_id	UUID	✅	—	References the student's enrollment
obtained_marks	DECIMAL(5,2)	❌	`NULL`	Marks obtained by the student
is_absent	BOOLEAN	✅	`FALSE`	Indicates whether the student was absent for the exam
is_published	BOOLEAN	✅	`FALSE`	Indicates whether this mark is visible to students and guardians
remarks	TEXT	❌	`NULL`	Optional remarks from the evaluator
entered_by	UUID	✅	—	User who entered the marks
entered_at	TIMESTAMPTZ	✅	`NOW()`	Date and time when marks were entered
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
exam_subject_id	exam_subjects(id)
student_enrollment_id	student_enrollments(id)

Column	References
entered_by	users(id)

Unique Constraints

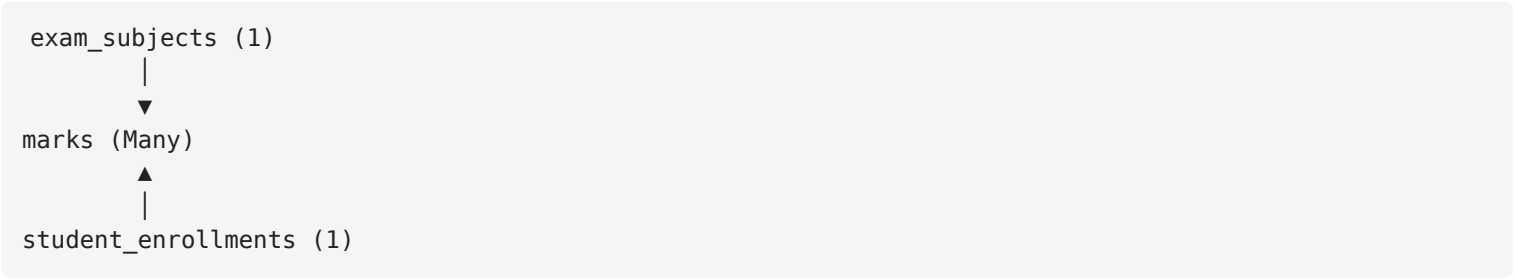
```
UNIQUE(exam_subject_id, student_enrollment_id)
```

A student can have only one mark record for a specific exam subject.

Recommended Indexes

- INDEX(exam_subject_id)
- INDEX(student_enrollment_id)
- INDEX(is_published)
- INDEX(is_absent)
- INDEX(entered_by)
- UNIQUE(exam_subject_id, student_enrollment_id)

Relationships



Each mark belongs to one exam subject and one student enrollment.

Business Rules

- Every mark belongs to exactly one exam subject.
- Every mark belongs to exactly one student enrollment.
- A student cannot have multiple marks for the same exam subject.
- `obtained_marks` cannot exceed the `full_marks` defined in `exam_subjects`.

- If `is_absent = TRUE`, then `obtained_marks` must be `NULL` (enforced by `CHECK(NOT (is_absent = TRUE AND obtained_marks IS NOT NULL))`).
- Marks may be entered before publication.
- Students and guardians can only view marks where `is_published = TRUE`.
- Updating marks after publication requires appropriate permissions and must be recorded in `audit_logs`.

Example

```
 Final Examination

Mathematics

Full Marks: 100

---------------------------------------

Student A
Obtained: 85

Student B
Obtained: 67

Student C
Absent
```

Mark Entry Workflow

```
 Create Examination
        │
        ▼
Configure Subjects
        │
        ▼
Teachers Enter Marks
        │
        ▼
Review Marks
        │
        ▼
Publish Marks
        │
        ▼
Students & Guardians Can View Results
```

Notes

Why reference `student_enrollments` instead of `students`?

Marks belong to a student's academic enrollment.

Example:

```
 Student

↓

2026
Class 6
↓

Marks

↓

2027
Class 7
↓

New Marks
```

This preserves the student's complete academic history across promotions.

Why not store Grade, GPA, Percentage or Position?

These values are **calculated**, not entered by users.

Examples:

```
 Obtained Marks
        │
        ▼
Grade Scale
        │
        ▼
Grade
        │
        ▼
GPA
```

If grading policies change, reports automatically reflect the new rules without updating stored records.

Why use `DECIMAL(5,2)` for marks?

Some schools award fractional marks.

Examples:

```
 18.50

24.75

79.25
```

Using `DECIMAL` makes the system flexible without requiring future schema changes.

Why have `is_absent`?

A score of **0** and an **absence** are not the same.

Example:

```
 Student A: Obtained Marks = 0, is_absent = FALSE
Meaning: Student attended but scored zero.

Student B: Obtained Marks = NULL, is_absent = TRUE
Meaning: Student did not attend the examination.
```

Keeping these separate improves reporting accuracy.

Why have `is_published`?

Teachers often enter marks over several days.

This field allows marks to remain hidden until they have been reviewed and officially released.

Benefits:

- Prevents students from seeing incomplete marks.
- Supports review and correction before publication.
- Allows gradual mark entry while controlling visibility.

Future Expansion

The table can be safely extended later with fields such as:

- moderated_marks
- practical_obtained_marks
- theory_obtained_marks

- approved_by
- approved_at
- evaluation_status

without requiring any structural redesign.

Table 20 — grade_scales

Purpose

Stores the grading policy for a tenant.

A grade scale defines how obtained marks are converted into grades and grade points (GPA). Unlike hardcoded grading systems, this table allows each school to configure its own grading policy without changing the application code.

The application calculates grades dynamically using this table whenever results, report cards, or transcripts are generated.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant
name	VARCHAR(100)	✅	—	Grade scale name (e.g., Default Grade Scale)
min_marks	DECIMAL(5,2)	✅	—	Minimum marks for this grade
max_marks	DECIMAL(5,2)	✅	—	Maximum marks for this grade
grade_letter	VARCHAR(20)	✅	—	Grade label (e.g., A+, A, B, Excellent)
grade_point	DECIMAL(3,2)	✅	—	Grade point (GPA)
remarks	VARCHAR(100)	❌	`NULL`	Optional description of the grade
display_order	INTEGER	✅	—	Order in which grades are displayed
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)

Unique Constraints

```
UNIQUE(tenant_id, name, grade_letter)
```

Each grade letter must be unique within a grading scale.

Recommended Indexes

- INDEX(tenant_id)
- INDEX(display_order)
- INDEX(min_marks)
- INDEX(max_marks)

Relationships

```
 tenants (1)
       │
       ▼
grade_scales (Many)

marks
       │
       ▼
Application calculates
       │
       ▼
Grade & GPA
```

The `marks` table does **not** reference `grade_scales` directly. Grades are calculated dynamically by the application.

Business Rules

- Every grade belongs to exactly one tenant.
- A tenant may define multiple grading scales.
- `min_marks` must be less than or equal to `max_marks`.
- Mark ranges must not overlap within the same grading scale.
- `grade_point` cannot be negative.
- The application should validate that the grading ranges cover the required mark range without gaps if required by the school's policy.
- Grades are calculated dynamically when generating reports and transcripts.
- Updating a grading scale affects future grade calculations unless historical grades are stored separately (not required for MVP).

Example

```
 Default Grade Scale

Marks        Grade    GPA

80–100       A+       5.00

70–79        A        4.00

60–69        A-       3.50

50–59        B        3.00

40–49        C        2.00

33–39        D        1.00

0–32         F        0.00
```

Grade Calculation Example

```
 Student Marks

Mathematics

Obtained Marks = 76

↓

Application searches grade_scales

↓

70–79
```

```
↓

Grade = A

GPA = 4.00
```

Grade calculation occurs at runtime and is **not stored** in the `marks` table.

Notes

Why doesn't the `marks` table store grades?

Grades are derived from obtained marks and the grading policy.

Example:

```
 Obtained Marks
        │
        ▼
Grade Scale
        │
        ▼
Grade
        │
        ▼
GPA
```

Storing grades would duplicate data and create inconsistencies if the grading policy changes.

Why have a `name` field?

Although most schools will use only one grading scale in the MVP, this field allows future support for multiple grading systems.

Examples:

- Default Grade Scale
- SSC Grade Scale
- Cambridge Grade Scale
- College Grade Scale

This adds flexibility without increasing complexity.

Why use `display_order`?

Schools may want grades displayed in a specific order.

Example:

```
A+

A

A-

B

C

D

F
```

Using `display_order` avoids relying on alphabetical sorting.

Why use `DECIMAL` for mark ranges?

Some schools use fractional marks.

Example:

```
79.50

64.25

32.75
```

Using `DECIMAL` ensures accurate grading without requiring future schema changes.

Future Expansion

The table can be safely extended later with fields such as:

- academic_year_id
- is_default
- effective_from
- effective_to
- grading_type (Percentage / GPA / Letter)
- created_by

without requiring any structural redesign.

📁 Communication

Table 21 — notices

Purpose

Stores school notices that are published as **PDF** or **Image** files.

A notice is a file-based announcement uploaded by the School Admin or an authorized Manager. Once published, it becomes visible to **everyone within the tenant (subdomain)**.

This table stores only the notice metadata. The actual file is stored in external storage (e.g., Local Storage, AWS S3, Cloudinary, Supabase Storage).

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the tenant
academic_year_id	UUID	❌	`NULL`	References the academic year (nullable for year-independent notices)
title	VARCHAR(255)	✅	—	Notice title
file_url	TEXT	✅	—	URL or path of the uploaded file
file_type	NOTICE_FILE_TYPE	✅	—	Type of uploaded file
publish_at	TIMESTAMPTZ	✅	`NOW()`	Date and time when the notice becomes visible
expires_at	TIMESTAMPTZ	❌	`NULL`	Optional expiration date after which the notice is hidden
is_published	BOOLEAN	✅	`TRUE`	Determines whether the notice is currently visible
uploaded_by	UUID	✅	—	User who uploaded the notice
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)
academic_year_id	academic_years(id)
uploaded_by	users(id)

NOTICE_FILE_TYPE ENUM

```
PDF

IMAGE
```

Recommended Indexes

- INDEX(tenant_id)
- INDEX(is_published)
- INDEX(publish_at)
- INDEX(expires_at)
- INDEX(file_type)

Relationships

```
tenants (1)
      │
      ▼
notices (Many)
      ▲
      │
uploaded_by (1) → users
```

Each notice belongs to one tenant and is uploaded by one user.

Business Rules

- Every notice belongs to exactly one tenant.
- Every notice must have a title.
- Every notice must contain exactly one uploaded file.
- Supported file types are **PDF** and **IMAGE** only.
- All authenticated users within the tenant can view published notices.
- A notice becomes visible to users when `publish_at` is reached and `is_published = TRUE`.
- If `expires_at` is set and the date has passed, the notice is automatically hidden from users.
- Deleting a notice should also remove its associated file from storage.
- Updating a notice must be recorded in the `audit_logs` table.

Notice Workflow

```
 School Admin / Manager
        │
        ▼
Upload PDF or Image
        │
        ▼
Save Notice
        │
        ▼
(Optional)
Schedule Publication
        │
        ▼
Published
        │
        ▼
Visible to Everyone in the Tenant
        │
        ▼
(Optional)
Automatically Hidden After Expiration
```

Example

```
 Title:
Final Examination Routine

File:
final_exam_routine.pdf

Publish:
```

```
01 Dec 2026

Expires:
20 Dec 2026
```

Notes

Why store only the file instead of notice content?

The MVP is designed around file-based notices.

Schools typically publish:

- Examination Routine (PDF)
- Holiday Notice (PDF)
- Admission Circular (Image)
- Fee Notice (PDF)

This approach is simpler and matches real-world usage.

Why no `audience` column?

Every published notice is visible to **all authenticated users within the school's subdomain**.

Since the audience is always the same, storing an `audience` field would add unnecessary complexity.

If audience-specific notices are needed in the future, it can be added without affecting the existing design.

Why use `publish_at`?

This allows notices to be scheduled in advance.

Example:

```
 Today:
Upload Exam Routine

Tomorrow 8:00 AM:
Automatically Publish
```

No manual intervention is required.

Why use `expires_at`?

Many notices are temporary.

Examples:

- Examination Routine
- Holiday Announcement
- Admission Circular

Once expired, the system can automatically hide the notice without deleting it.

Future Expansion

The table can be safely extended later with fields such as:

- thumbnail_url
- download_count
- file_size
- mime_type
- version
- published_by

without requiring any structural redesign.

Table 22 — notifications

Purpose

Stores every outbound notification sent by the system.

Unlike the `notices` table, which displays announcements inside the application, this table records communications that are **actively sent** to recipients through external channels such as SMS or Email.

This table also acts as a delivery log, allowing administrators to monitor notification history, retry failed messages, and troubleshoot delivery issues.

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key

Column	Type	Required	Default	Description
tenant_id	UUID	✅	—	References the tenant
channel	NOTIFICATION_CHANNEL	✅	—	Notification delivery channel
recipient	VARCHAR(255)	✅	—	Destination phone number or email address
subject	VARCHAR(255)	❌	`NULL`	Email subject (used only for Email)
message	TEXT	✅	—	Notification message content
status	NOTIFICATION_STATUS	✅	`PENDING`	Current delivery status
retry_count	INTEGER	✅	`0`	Number of delivery retry attempts
provider_response	TEXT	❌	`NULL`	Response from the SMS/Email provider
trigger_type	NOTIFICATION_TRIGGER	✅	`MANUAL`	Source that triggered the notification
reference_id	UUID	❌	`NULL`	Related entity ID based on the trigger type
sent_by	UUID	❌	`NULL`	User who initiated the notification (NULL if automated)
sent_at	TIMESTAMPTZ	❌	`NULL`	Date and time when the notification was successfully sent
created_at	TIMESTAMPTZ	✅	`NOW()`	Record creation timestamp
updated_at	TIMESTAMPTZ	✅	`NOW()`	Last update timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)
sent_by	users(id)

> **Note:** `reference_id` is intentionally **not** defined as a database foreign key because it may reference different tables depending on `trigger_type`.

NOTIFICATION_CHANNEL ENUM

```
SMS

EMAIL
```

NOTIFICATION_STATUS ENUM

```
PENDING

SENT

FAILED
```

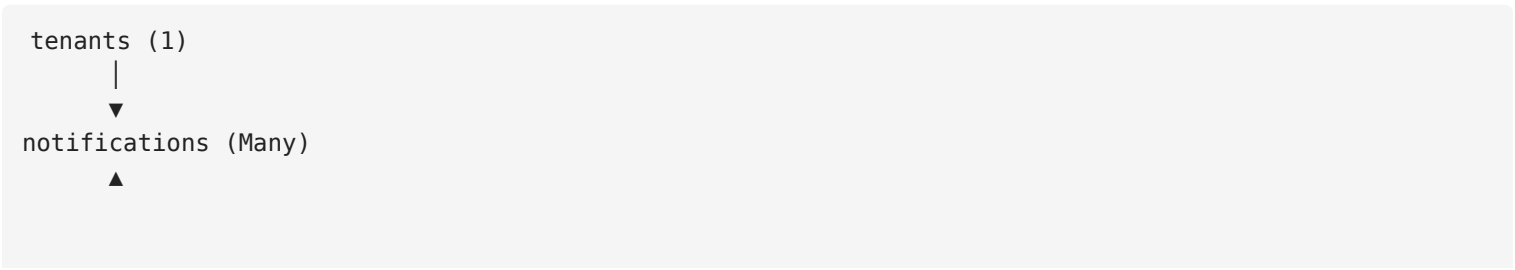
NOTIFICATION_TRIGGER ENUM

```
MANUAL

ATTENDANCE

RESULT

SYSTEM
```

Recommended Indexes

- INDEX(tenant_id)
- INDEX(channel)
- INDEX(status)
- INDEX(trigger_type)
- INDEX(sent_at)
- INDEX(created_at)

Relationships



```
      |
users (1) (sent_by)
```

Notifications belong to one tenant and may optionally be initiated by a user.

Business Rules

- Every notification belongs to exactly one tenant.
- Every notification must have a delivery channel.
- Every notification must have one recipient.
- SMS notifications ignore the `subject` field.
- Email notifications may include a subject.
- The original notification content must always be stored.
- Failed notifications should remain in the database for troubleshooting.
- `retry_count` increases each time the system retries delivery.
- `provider_response` stores the response returned by the SMS or Email provider.
- `reference_id` identifies the business record that triggered the notification.
- Automatic notifications may have `sent_by = NULL`.
- Notification history should never be deleted as part of normal system operations.

Notification Workflow

```
 System Event
        |
        ▼
Create Notification Entry
        |
        ▼
Status = PENDING
        |
        ▼
Attempt Delivery
        |
        ├── SUCCESS
        |       |
        |       ▼
        |   Status = SENT
        |   sent_at = NOW()
        |
        └── FAILURE
                |
                ▼
            Status = FAILED
            retry_count += 1
            provider_response stored
```

Example

```
 Channel: SMS
Recipient: +8801712345678
Trigger: ATTENDANCE
Status: SENT
Sent At: 10 Jul 2026 10:25 AM
```

Notes

Why store notifications?

This table serves as a permanent delivery history.

It allows administrators to answer questions such as:

- Was the SMS actually sent?
- Who received it?
- When was it sent?
- Why did it fail?
- Can it be retried?

Why store the complete message?

Messages should never be regenerated.

Example:

```
 Dear Parent,

Your child was absent today.
```

Keeping the original message preserves an accurate communication history.

Why use `trigger_type` and `reference_id`?

These fields allow notifications to be linked back to the business event that generated them.

Examples:

Trigger	Reference
ATTENDANCE	attendance_session_id
RESULT	exam_id
MANUAL	NULL
SYSTEM	NULL

This makes debugging and reporting much easier.

Why store `provider_response`?

External providers often return useful information.

Examples:

```
 Message Accepted

Invalid Destination Number

Authentication Failed

Daily Limit Exceeded
```

Keeping these responses greatly simplifies troubleshooting.

Why keep failed notifications?

Failed notifications are valuable for:

- Retry mechanisms
- Error investigation
- Delivery reporting
- Support requests

Deleting failed notifications would remove important operational history.

Future Expansion

The table can be safely extended later with fields such as:

- scheduled_at
- delivered_at
- read_at

- provider_name
- external_message_id
- attachment_url
- priority

without requiring any structural redesign.

📁 System

Table 23 — audit_logs

Purpose

Stores a permanent history of important actions performed by users inside the system.

This table provides accountability and transparency by recording:

- Who performed an action
- What action was performed
- Which module was affected
- Which record was changed
- What data changed
- When the action happened

This table helps administrators answer questions such as:

- Who deleted this student?
- Who edited attendance?
- When was a result modified?
- Who changed the SMS balance?
- Who deactivated a tenant?
- Who changed user permissions?

Without an audit log system, tracking important changes becomes impossible.

Should We Log Everything?

No.

The system should **not** log every database read operation or every API request.

Only important business actions should be recorded.

Examples:

- Student created
- Student updated
- Student deleted
- Attendance edited
- Marks updated
- Notice deleted
- SMS balance changed
- Tenant deactivated
- User permission changed

Schema

Column	Type	Required	Default	Description
id	UUID	✅	`gen_random_uuid()`	Primary Key
tenant_id	UUID	✅	—	References the school/tenant
user_id	UUID	❌	`NULL`	User who performed the action
module	AUDIT_MODULE	✅	—	System module where action occurred
action	AUDIT_ACTION	✅	—	Type of action performed
entity_type	VARCHAR(100)	✅	—	Type of affected entity
entity_id	UUID	❌	`NULL`	ID of the affected record
description	TEXT	❌	`NULL`	Human-readable explanation of the action
old_values	JSONB	❌	`NULL`	Previous data before update
new_values	JSONB	❌	`NULL`	New data after update
ip_address	VARCHAR(45)	❌	`NULL`	Client IP address
user_agent	TEXT	❌	`NULL`	Browser/device information
created_at	TIMESTAMPTZ	✅	`NOW()`	Action timestamp

Primary Key

```
id
```

Foreign Keys

Column	References
tenant_id	tenants(id)
user_id	users(id)

Unique Constraints

None needed for audit_logs. Each audit entry is a unique event and does not require uniqueness enforcement beyond the primary key.

Recommended Indexes

- INDEX(tenant_id)
- INDEX(user_id)
- INDEX(module)
- INDEX(action)
- INDEX(entity_type)
- INDEX(entity_id)
- INDEX(created_at)

Relationships

```
 tenants (1)
      |
      ▼
audit_logs (Many)
      ▲
      |
users (1)
```

Each audit log entry belongs to one tenant and is optionally associated with one user.

AUDIT_MODULE ENUM

```
 STUDENT_MANAGEMENT
ATTENDANCE_MANAGEMENT
RESULT_MANAGEMENT
NOTICE_BOARD
NOTIFICATIONS
USER_MANAGEMENT
```

```
TENANT_MANAGEMENT
SYSTEM
```

> New modules can be added later without affecting existing audit records.

AUDIT_ACTION ENUM

```
 CREATED
UPDATED
DELETED
PUBLISHED
UNPUBLISHED
APPROVED
REJECTED
PERMISSION_CHANGED
SMS_BALANCE_CHANGED
TENANT_STATUS_CHANGED
USER_STATUS_CHANGED
LOGIN
LOGOUT
```

Business Rules

- Every audit log entry belongs to exactly one tenant.
- `user_id` is `NULL` only for system-generated actions.
- `module` and `action` must use predefined ENUM values.
- `entity_type` should use a clear, consistent naming convention for the affected table or resource.
- `old_values` and `new_values` should be populated for UPDATE actions.
- For DELETE actions, only `old_values` should be populated.
- For CREATE actions, only `new_values` should be populated.
- Audit logs must be **append-only**. Existing records must never be modified or deleted.
- Routine read operations and simple queries must not be logged.
- Audit log retention and archival should be managed by the application.

Notes

Why log to the database instead of a file?

Database logging:

- Integrates with the application's backup and retention policies.

- Is searchable and filterable.
- Supports multi-tenancy.
- Allows administrators to view audit history through the application.

Why use JSONB for `old_values` and `new_values`?

JSONB provides:

- Flexible schema — different entity types can store different fields.
- Query support — PostgreSQL allows querying inside JSONB data.
- Easy diffs — the application can compare old and new values programmatically.

Why not log every request?

Logging every request would cause:

- Excessive storage usage.
- Slower application performance.
- Difficulty finding important events in a sea of routine requests.

Only important business actions are worth recording.

Why make `user_id` optional?

Some system events happen without a user context.

Examples:

- Automatic tenant deactivation after failed payment.
- Scheduled system task execution.
- Internal system migration.

These events should still be logged for accountability.

Why use ENUMs for `module` and `action`?

ENUMs ensure:

- Consistent naming across all audit entries.
- Simple filtering in queries.
- No risk of typos or inconsistent values.

Future Expansion

The table can be safely extended later with fields such as:

- session_id
- request_id
- correlation_id
- duration_ms
- affected_record_count

without requiring any structural redesign.